

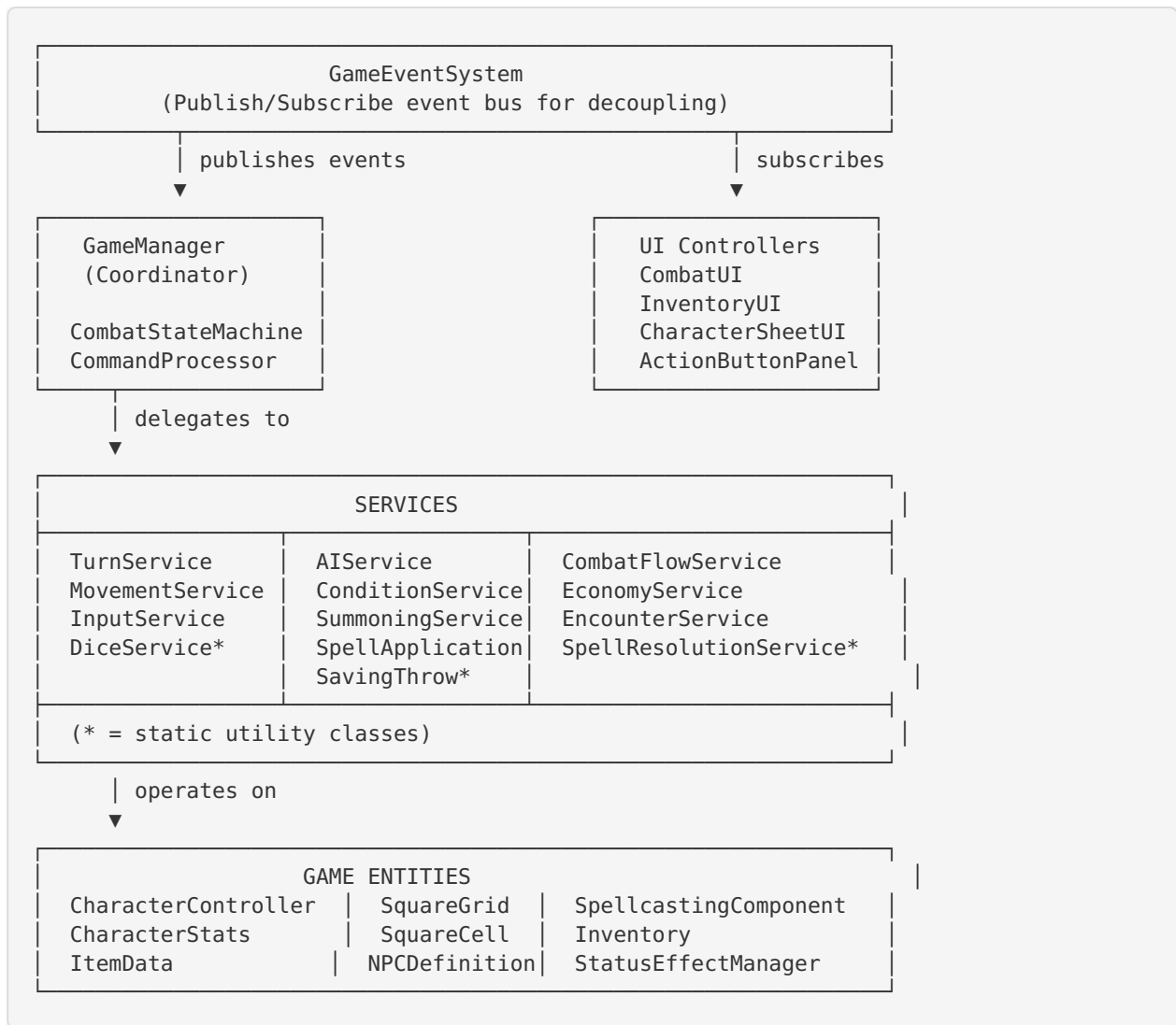
D&D 3.5e Prototype — Refactoring Guide

This document provides patterns and instructions for continuing the codebase refactoring. It covers service extraction from `GameManager`, advanced architectural patterns, magic folder reorganization, and general code quality practices.

Table of Contents

1. [Architecture Overview](#)
 2. [GameManager Responsibility Breakdown](#)
 3. [Service Extraction Pattern](#)
 4. [Event-Driven Architecture \(GameEventSystem\)](#)
 5. [Command Pattern \(CommandProcessor\)](#)
 6. [Combat State Machine](#)
 7. [How to Extract More Services from GameManager](#)
 8. [Magic Folder Migration Pattern](#)
 9. [Code Quality Best Practices](#)
 10. [Completed Refactoring Summary](#)
-

1. Architecture Overview



Key Architectural Decisions

- GameManager as Coordinator:** GameManager delegates to services but retains orchestration responsibility
- Partial Classes for Organization:** GameManager is split across 14 partial class files by domain
- Event System for Decoupling:** GameEventSystem allows UI to react to game state changes without direct coupling
- Command Pattern for Actions:** CommandProcessor provides a unified pipeline for all game actions
- State Machine for Combat Flow:** CombatStateMachine models combat phases explicitly

2. GameManager Responsibility Breakdown

Current Partial Class Files (Post-Refactoring)

File	Lines	Domain
GameManager.cs	~10,300	Core: setup, party, turn flow, UI orchestration, conditions, movement
GameManager.SpellCasting.cs	~6,350	Spell slot consumption, targeting, cast execution, buff/duration tracking, concentration
GameManager- ager.CombatActions.cs	~2,250	Player attack targeting, special attacks, cell click routing, attack execution
GameManager.TestConfigs.cs	~1,875	Test encounter party configurations (20+ test scenarios)
GameManager_NewSpells.cs	~1,600	Spell resolution: Lightning Bolt, Fireball, Hold Person, Blink, etc.
GameManager.NPCTurns.cs	~1,430	NPC AI turn execution, adaptive retargeting, special attacks
GameManager_Grease.cs	~860	Grease spell targeting and effects
GameManager_MirrorImage.cs	~825	Mirror Image spell mechanics
GameManager.NPCSetup.cs	~710	NPC spawning, initialization, AI profile assignment
GameManager- ager_FlamingSphere.cs	~675	Flaming Sphere spell mechanics
GameManager- ager.LootCollection.cs	~670	Post-combat loot, XP awards, level-up
GameManager- ager.DispelCounterspell.cs	~600	Dispel Magic & Counterspell systems
GameManager- ager_ConcealmentAreas.cs	~430	Concealment area spells (Fog Cloud, Darkness, etc.)
GameManager- ager.CombatFlowAccessors.cs	~130	Accessor methods for CombatFlowService

Remaining Responsibilities in Main File (~10,300 lines)

Category	Est. Lines	Description
Party/Character Setup	~1,800	Character creation, equipment setup, class defaults
Turn Management	~800	Turn start/end, initiative, round management
UI Orchestration	~1,500	Button handlers, action menus, pre-combat hub
Condition Handling	~800	Charm/fascination/invisibility breaking, condition expiry
Inventory/Item Actions	~600	Consumables, drop/pickup, reload
Movement	~500	Movement range display, path preview, AoO confirmation
Combat Victory	~400	Victory detection, XP tracking
Input Handling	~400	Cell clicks, hover tooltips, input routing
Fields/Properties	~800	State fields, service references, constants
Misc (daily effects, etc.)	~700	Disease/poison, rage, emanations

3. Service Extraction Pattern

Template: EconomyService

```
// Assets/Scripts/Services/YourNewService.cs
public class YourNewService : MonoBehaviour
{
    private GameManager _gameManager;

    public void Initialize(GameManager gameManager)
    {
        _gameManager = gameManager;
    }

    public void Cleanup()
    {
        // Reset state on combat end or destruction.
    }

    // ... extracted methods go here
}
```

Existing Services (16 total)

Service	Type	Responsibility
TurnService	MonoBehaviour	Initiative order, turn progression
MovementService	MonoBehaviour	Grid movement, pathfinding, AoO
InputService	MonoBehaviour	Mouse/keyboard input, UI filtering
ConditionService	MonoBehaviour	Combat conditions tracking
AIService	MonoBehaviour	NPC AI decisions, target selection
CombatFlowService	MonoBehaviour	Attack pipelines, hit resolution
EconomyService	MonoBehaviour	Gold, stash, store, buy/sell
SummoningService	MonoBehaviour	Summoned creature lifecycle
EncounterService	MonoBehaviour	Encounter lifecycle, enemy tracking
SpellApplicationService	MonoBehaviour	Spell effect application, tracking
DiceService	Static	Dice rolling (d20, d6, etc.)
SpellResolutionService	Static	Spell failure, SR checks
SavingThrowResolver	Static	Saving throw calculations
GameEventSystem	Singleton	Event bus for pub/sub decoupling
CommandProcessor	MonoBehaviour	Unified action execution pipeline
CombatStateMachine	Plain class	Combat phase state management

4. Event-Driven Architecture (GameEventSystem)

Overview

`GameEventSystem` (`Assets/Scripts/Core/GameEventSystem.cs`) is a publish-subscribe event bus that decouples game logic from UI. Instead of `GameManager` directly calling UI methods, it publishes events that UI controllers can subscribe to independently.

Usage

```
// Publishing an event (from GameManager or services):
GameEventSystem.Instance.Publish(new CombatStartedEvent
{
    TotalCombatants = 8,
    Round = 1
});

// Subscribing to an event (from UI controllers):
void OnEnable()
{
    GameEventSystem.Instance.Subscribe<CombatStartedEvent>(OnCombatStarted);
}

void OnDisable()
{
    GameEventSystem.Instance.Unsubscribe<CombatStartedEvent>(OnCombatStarted);
}

private void OnCombatStarted(CombatStartedEvent evt)
{
    // Update UI...
}
```

Available Events

Combat Flow Events

- `CombatStartedEvent` — Combat begins
- `CombatEndedEvent` — Victory or defeat
- `NewRoundEvent` — New round begins
- `TurnStartedEvent` — Character's turn starts
- `TurnEndedEvent` — Character's turn ends
- `CombatStateChangedEvent` — State machine transition

Character State Events

- `DamageTakenEvent` — Damage received
- `HealingReceivedEvent` — Healing received
- `CharacterDefeatedEvent` — Character killed
- `ConditionAppliedEvent` — Condition added
- `ConditionRemovedEvent` — Condition removed

Combat Action Events

- `AttackResolvedEvent` — Attack hit/miss
- `SpellCastEvent` — Spell cast
- `SpecialAttackEvent` — Special attack result

- `CommandExecutedEvent` — Command processed

Economy/Inventory Events

- `GoldChangedEvent` — Party gold changed
- `ItemEquippedEvent` — Equipment change
- `ConsumableUsedEvent` — Potion/scroll used

Movement/UI Events

- `CharacterMovedEvent` — Grid movement
- `UIPhaseChangedEvent` — UI phase transition
- `ShowActionChoicesEvent` — Action menu display
- `StatsUIRefreshRequestedEvent` — Stats refresh

Adding New Events

1. Define a struct implementing `IGameEvent` :

```
public struct MyNewEvent : IGameEvent
{
    public string SomeData;
    public int SomeValue;
}
```

1. Publish from game logic: `GameEventSystem.Instance.Publish(new MyNewEvent { ... });`
2. Subscribe from UI: `GameEventSystem.Instance.Subscribe<MyNewEvent>(handler);`

5. Command Pattern (CommandProcessor)

Overview

`CommandProcessor` (`Assets/Scripts/Core/Commands/CommandProcessor.cs`) provides a unified pipeline for executing game actions. Every player/AI action flows through validation → execution → logging.

Benefits

- **Consistent validation** before every action
- **Action history** for debugging and future replay
- **Pre/post execution hooks** for cross-cutting concerns
- **Command queue** for AI batch execution
- **Future undo/redo** support

Interface

```
public interface IGameCommand
{
    string DisplayName { get; }
    CharacterController Actor { get; }
    bool CanExecute(out string reason);
    void Execute();
    ActionCostType ActionCost { get; }
}

// For async commands (movement, animations):
public interface IGameCommandAsync : IGameCommand
{
    IEnumerator ExecuteAsync();
}
```

Available Commands

- `AttackCommand` — Standard single attack
- `FullAttackCommand` — Full-round attack
- `MoveCommand` — Movement (async)
- `FiveFootStepCommand` — 5-foot step
- `CastSpellCommand` — Spell casting
- `UseItemCommand` — Consumable use
- `EndTurnCommand` — End turn

Creating New Commands

```
public class MyNewCommand : IGameCommand
{
    public string DisplayName => "My Action";
    public CharacterController Actor { get; }
    public ActionCostType ActionCost => ActionCostType.Standard;

    public MyNewCommand(CharacterController actor) { Actor = actor; }

    public bool CanExecute(out string reason)
    {
        // Validate action is legal
        reason = null;
        return true;
    }

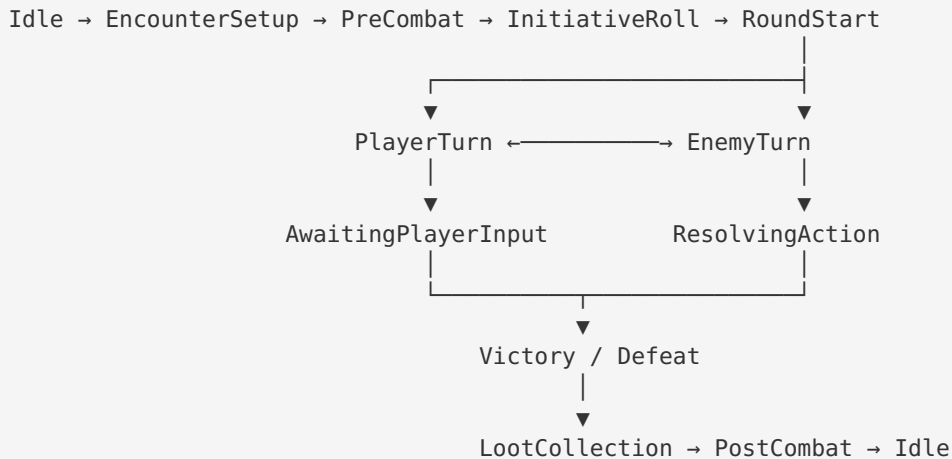
    public void Execute()
    {
        // Perform the action
    }
}
```

6. Combat State Machine

Overview

`CombatStateMachine` (`Assets/Scripts/Combat/CombatStateMachine.cs`) models combat as explicit states with validated transitions, replacing implicit phase tracking through boolean flags.

States



Player Input Sub-States

When in `AwaitingPlayerInput` :

- `ChoosingAction` — Main action menu
- `SelectingAttackTarget` — Choosing attack target
- `SelectingSpellTarget` — Choosing spell target
- `SelectingMovement` — Choosing movement destination
- `SelectingAoEPlacement` — Placing AoE spell
- `SelectingSpecialTarget` — Choosing special attack target
- `PlacingSummon` — Placing summoned creature
- `ConfirmingAction` — AoO confirmation prompt

Usage

```

// Transition between states:
GameManager.Instance.CombatState.TransitionTo(CombatStateMachine.CombatState.PlayerTurn);

// Query current state:
if (CombatState.IsAwaitingPlayerInput) { ... }
if (CombatState.IsCombatActive) { ... }

// Listen for state changes:
CombatState.OnStateChanged += (oldState, newState) =>
{
    Debug.Log($"Combat: {oldState} → {newState}");
};
  
```

7. How to Extract More Services from GameManager

Recommended Next Extractions (Priority Order)

A. SpellCastingService (Highest Impact)

What to extract from `GameManager.SpellCasting.cs` :

- Spell targeting logic
- Spell slot consumption
- AoE targeting and preview
- Concentration tracking
- **~3,000-4,000 lines** (keep spell resolution methods in their own partial classes)

B. PartyManagementService

What to extract from main `GameManager.cs` :

- Party composition, member management
- Character creation completion
- Party stash management
- Team queries (IsPC, IsEnemyTeam, GetTeamMembers)
- **~800-1,000 lines**

C. UIOrchestrationService

What to extract from main `GameManager.cs` :

- ShowActionChoices and all button visibility logic
- Pre-combat hub phase management
- Inventory/store/skills/character sheet window management
- Hover tooltip management
- **~1,200-1,500 lines**

D. RestService

What to extract:

- RestorePartyAfterCombat
- ProcessDailyEffects
- Disease/poison management
- **~300-500 lines**

Step-by-Step Checklist for Any Extraction

1. [] **Identify** all related fields, properties, and methods in GameManager
 2. [] **Create** new service file in `Assets/Scripts/Services/`
 3. [] **Copy** the identified code into the service (don't cut yet)
 4. [] **Add** `Initialize()` with all required dependencies
 5. [] **Add** the `[SerializeField]` field in GameManager
 6. [] **Wire** initialization in `Awake()`
 7. [] **Update** GameManager methods to delegate to the service
 8. [] **Add event publishing** for key state changes
 9. [] **Test** that everything compiles and works in Unity
 10. [] **Remove** dead fallback code once verified
 11. [] **Commit** with descriptive message
-

8. Magic Folder Migration Pattern

New Folder Structure

Assets/Scripts/Magic/

Spells/Databases/

AreaEffects/

StatusEffects/

SpellDatabase.cs

SpellData.cs

[enums and data classes]

SpellDatabase_*.cs files

*AreaEffect.cs files

*EffectData.cs files

Main coordinator (stays here)

Core data structure (stays here)

Stay in root

How to Migrate

1. Move the file to its new folder (no code changes needed)
2. Delete the old .meta file if needed
3. Verify compilation in Unity

9. Code Quality Best Practices

- **DiceService for all rolls:** Use `DiceService.D20()` , `DiceService.Roll()` , etc.
- **AttackCalculator for feat modifiers:** Centralize feat-based calculations
- **Consistent logging:** Use `[Category]` prefixes in `Debug.Log`
- **Null safety:** Always null-check Unity objects
- **Event-driven communication:** Use `GameEventSystem` for cross-system events
- **Command pattern for actions:** Use `CommandProcessor` for player/AI actions

Naming Conventions

Type	Convention	Example
Services	*Service.cs	EconomyService.cs
Static utilities	Descriptive name	DiceService.cs
Managers	*Manager.cs	AreaEffectManager.cs
Data classes	*Data.cs	SpellData.cs
UI classes	*UI.cs	StoreUI.cs
Systems	*System.cs	GrappleSystem.cs
Commands	*Command.cs	AttackCommand.cs
Events	*Event struct	CombatStartedEvent

10. Completed Refactoring Summary

Phase 1: Core Service Extraction

- Extracted `DiceService` (static) for centralized dice rolling
- Extracted `AttackCalculator` for feat-based combat modifiers

Phase 2: Combat System Cleanup

- Extracted `SavingThrowResolver`, `SpellResolutionService`, `CombatFlowService`
- Extracted `MovementService`, `InputService`, `ConditionService`
- Extracted `TurnService`, `AIService`

Phase 3: Template Implementation

- Extracted `EconomyService` as template
- Created Magic folder structure

Phase 4: Service Expansion

- Extracted `SummoningService`, `EncounterService`, `SpellApplicationService`

Phase 5: Advanced Architecture (Current)

- **GameManager main file reduced:** 23,290 → 10,288 lines (**56% reduction, ~13,000 lines extracted**)
- **New partial class files created:**
 - `GameManager.SpellCasting.cs` (~6,350 lines) — Spell casting orchestration
 - `GameManager.CombatActions.cs` (~2,250 lines) — Player combat actions & targeting
 - `GameManager.TestConfigs.cs` (~1,875 lines) — Test encounter configurations
 - `GameManager.NPCTurns.cs` (~1,430 lines) — NPC AI turn execution
 - `GameManager.NPCSetup.cs` (~710 lines) — NPC spawning & initialization
 - `GameManager.DispelCounterspell.cs` (~600 lines) — Dispel & counterspell systems
- **Event-Driven Architecture:** `GameEventSystem` with 20+ event types for UI decoupling
- **Command Pattern:** `CommandProcessor` with `IGameCommand` / `IGameCommandAsync` interfaces
- 7 concrete commands: `Attack`, `FullAttack`, `Move`, `FiveFootStep`, `CastSpell`, `UseItem`, `EndTurn`
- **Combat State Machine:** `CombatStateMachine` with 14 states and validated transitions
- **Updated documentation:** Architecture diagram, responsibility breakdown, pattern guides

Future Work

- Complete Magic folder migration (44 remaining files)
- Extract `SpellCastingService` from `GameManager.SpellCasting.cs`
- Extract `PartyManagementService` from main `GameManager.cs`
- Extract `UIOrchestrationService` from main `GameManager.cs`
- Wire all action button handlers through `CommandProcessor`
- Subscribe UI controllers to `GameEventSystem` events
- Add unit tests for new architectural patterns